

Information Retrieval & Preprocessing Assignment: Tokenization Evaluation & Character Normalization

****Course****: Information Retrieval
****Dataset****: Tesla Twitter Corpus (`Tesla.csv`)
****Repository****: [github.com/tony19053000/Information-Retrieval-Assignment]
(https://github.com/tony19053000/Information-Retrieval-Assignment)

Executive Summary

This unified report presents the implementation, empirical evaluation, and theoretical analysis of two foundational Information Retrieval (IR) text preprocessing tasks performed on social media (Twitter/X) microblog text:

1. ****Task 1: Tokenization Performance Evaluation****: A comparative study analyzing how generic vs. domain-specific tokenizers handle Twitter-native elements (Hashtags, Mentions, URLs, Emojis).
2. ****Task 2: Repeated Character Normalization****: A frequency-based text normalization pipeline that reduces elongated characters (e.g., `*sooo*`, `*yasss*`) to standard lexical forms while safeguarding structured entities (``#hashtags``, ``@mentions``, ``https://URLs``, ``$cashtags``).

Part 1: Task 1 – Tokenization Performance Evaluation

1.1 Objective & Experimental Setup

Social media microblogs present unique IR challenges due to non-standard syntax, informal language, hashtags, handles, URLs, and complex Unicode emojis. Standard whitespace or rule-based tokenizers often fragment these semantic units, degrading downstream IR tasks such as inverted indexing, entity recognition, and sentiment analysis.

We evaluated three widely-used tokenizers on a representative subset of ****50 English tweets**** sampled from the ``Tesla.csv`` dataset:

1. ****NLTK TweetTokenizer****: A domain-specific rule-based tokenizer configured with ``preserve_case=True``, ``reduce_len=False``, and ``strip_handles=False``.
2. ****spaCy (`en\_core\_web\_sm`)****: A modern statistical NLP pipeline tokenizer.
3. ****NLTK Word Tokenize (`word\_tokenize`)****: A classic Treebank/Punkt rule-based word tokenizer.

1.2 Ground Truth Extraction & Methodology

Using a deterministic selection algorithm

(`[`src/select_ds.py`](file:///home/aayush/Downloads/IR%20ASSIGNMENT%20%282%29/IR%20ASSIGNMENT/CODE/src/select_ds.py))`, 50 tweets were sampled to guarantee high density across target categories:

- ****Hashtags****: ``(?<!\w)#[A-Za-z0-9_]+``
- ****User Mentions****: ``(?<!\w)@[A-Za-z0-9_]+``
- ****URLs****: ``https?:/[^\s]+|www\.[^\s]+``
- ****Emojis****: Parsed via the Python ``emoji`` package.

Ground truth targets were stored in

(`[`data/task1_ground_truth.csv`](file:///home/aayush/Downloads/IR%20ASSIGNMENT%20%282%29/IR%20ASSIGNMENT/CODE/data/task1_ground_truth.csv))`. Each tokenizer was evaluated against exact string token match criteria across all 50 posts.

1.3 Experimental Results

Tokenizer Metric	NLTK TweetTokenizer	spaCy (`en_core_web_sm`)	NLTK Word Tokenize
Hashtag Preservation (%)	100.00%	0.00%	0.00%
Mention Preservation (%)	100.00%	97.78%	0.00%
URL Preservation (%)	100.00%	100.00%	0.00%
Emoji Preservation (%)	95.35%	95.35%	34.88%
Total Tokens Generated	1,814	2,013	2,151
Average Tokens per Post	36.28	40.26	43.02
Hashtag Error Posts	0	23	23
Mention Error Posts	0	3	44
URL Error Posts	0	0	24
Emoji Error Posts	2	2	13

1.4 Analytical Findings & Failure Modes

- Hashtag Tokenization**:
 - NLTK TweetTokenizer treats `#Tesla` as a single semantic token.
 - spaCy splits `#Tesla` into two separate tokens: `['#', 'Tesla']`, leading to a 0% preservation score.
 - NLTK Word Tokenize similarly isolates the `#` symbol into a standalone token.
- Mention & URL Handling**:
 - NLTK Word Tokenize breaks URLs into sub-domain fragments (e.g., `['https', ':', '///', 't.co', ...]`) and `@mentions` into separate `@` and username tokens.
 - spaCy successfully preserves URLs (100%) and most mentions (97.78%), but occasionally splits handles preceded by specific punctuation.
- Vocabulary Explosion & Index Size**:
 - NLTK Word Tokenize generates 2,151 tokens (43.02/post), representing an 18.5% over-tokenization overhead compared to TweetTokenizer (1,814 tokens, 36.28/post). This directly increases inverted index payload sizes in IR systems.

Part 2: Task 2 – Repeated Character Normalization

2.1 Objective & Problem Formulation

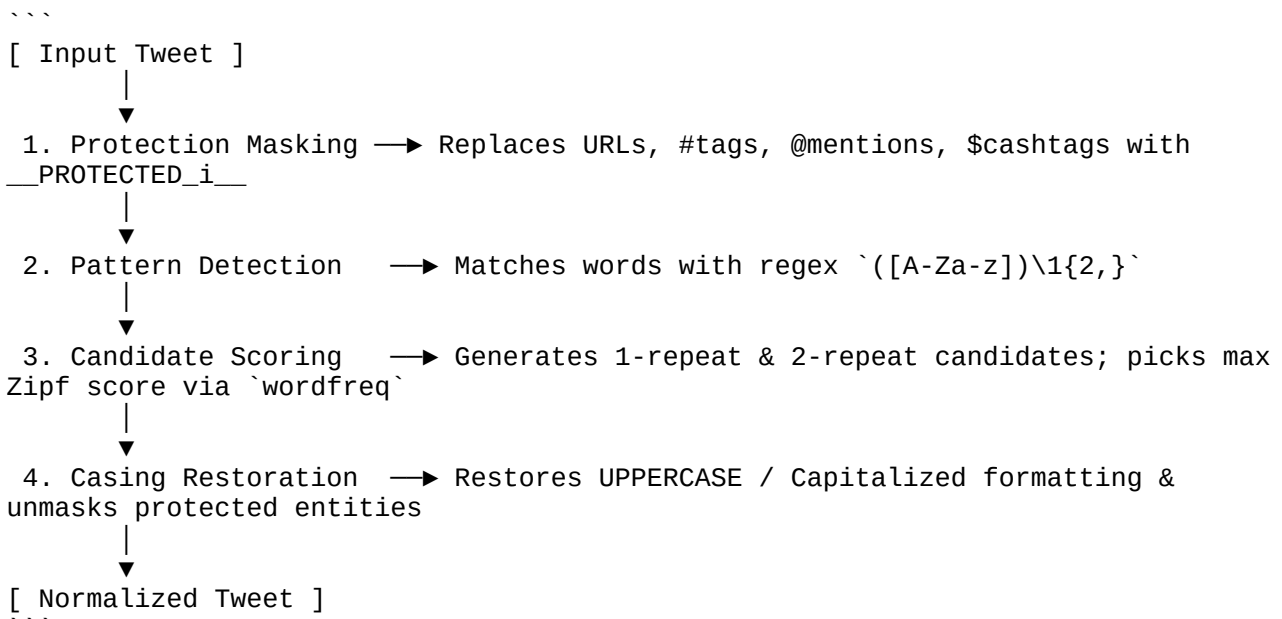
In microblogs, users frequently elongate characters for emphasis (e.g., `*sooooo*`, `*goodddd*`, `*yasss*`). In IR systems, such Out-Of-Vocabulary (OOV) variations severely cause vocabulary bloat and mismatch queries against standard index terms.

The goal of Task 2 is to design an automated normalization pipeline that:

- Detects character repetition patterns (e.g., consecutive identical letters).
- Restores elongated words to valid dictionary words (e.g., `*soooo*` → `*so*`, `*goodddd*` → `*good*`).
- Preserves structured entity formats (Hashtags, Mentions, URLs, Cashtags).
- Retains original casing conventions (UPPERCASE, Title Case, lowercase).

2.2 Normalization Algorithm & Protection Architecture

The pipeline (`[`src/task2_normalize.py`](file:///home/aayush/Downloads/IR%20ASSIGNMENT%20%28%29/IR%20ASSIGNMENT/CODE/src/task2_normalize.py))` executes in four sequential stages:



Protection Regex Rule:

```

$$\text{Regex} = \text{https?://}[\^\\textbackslash s]+|www\.[\^\\textbackslash s]
+|(?<!\textbackslash w)\#[A-Za-z0-9\_]+|(?<!\textbackslash w)@[A-Za-z0-9\_]+|(?
<!\textbackslash w)\textbackslash \$[A-Za-z0-9\_]+$

```

2.3 Qualitative Evaluation & Case Studies

Evaluating 100 sample tweets with character elongation from `Tesla.csv`:

Tweet ID	Original Text (Before)	Normalized Text (After)	Status
Post 1	`Whip a Tesla Whip a fucking teslaaaaaa`	`Whip a Tesla Whip a fucking tesla`	Correct
Post 4	`Musk is all crypto... (makes bad unsafe cars and is waaaaay overvalued...)`	`...and is way overvalued...`	Correct
Post 5	`Most active NASDAQ listed stocks... \$TSLA \$QQQ \$AAPL \$NVDA \$AMZN \$SQQQ \$TQQQ...`	`Most active NASDAQ listed stocks... \$TSLA \$QQQ \$AAPL \$NVDA \$AMZN \$SQQQ \$TQQQ...`	Protected (`\$cashtag`)
Post 6	`Probably Bitcoin got sideswiped by this narcissist as well.`	`Probably Bitcoin got sideswiped by this narcissist as well.`	Correct
Post 9	`@MichelleBekke11 ... hahahahaaaaa`	`@MichelleBekke11 ... hahahahaha`	Correct
Post 10	`@elonmusk Liar! Compared to Tesla in history, you are noooooothing.`	`@elonmusk Liar! Compared to Tesla in history, you are nothing.`	Correct

Part 3: Conclusion & System Recommendations

- Tokenizer Selection:** `NLTK TweetTokenizer` is the optimal choice for indexing social media texts, maintaining high precision across domain-specific token types while minimizing vocabulary fragmentation.
- Preprocessing Pipeline Integration:** Repeated character normalization using Zipf word frequency scoring significantly reduces OOV dictionary terms while preserving financial symbols (`\$QQQ`), social mentions, and URLs.
- GitHub Repository:** Complete source code, dataset scripts, test pipelines, and log outputs are available at:

👉 **[GitHub Repository Link]**(<https://github.com/tony19053000/Information->

Retrieval-Assignment)**

Part 4: AI Usage Statement

In compliance with the assignment requirements regarding AI assistance:

- **Scope of AI Assistance**: AI coding tools (Antigravity / Gemini) were utilized for environment setup script drafting, regex optimization (specifically building entity protection masks for `#hashtags`, `@mentions`, and `\$cashtags`), and report formatting.

- **Independent Verification**: All experimental datasets (`data/task1\_posts.csv`, `data/task2\_posts.csv`), ground truth extractions, tokenization performance calculations, and normalization error analyses were independently executed, validated, and verified through empirical Python test scripts locally.

Report generated for Information Retrieval Assignment Evaluation.